

Kabul University
Computer Science Faculty
Information System Department

JavaScript Lecture#2nd

Mohammad Zafar Shafaq

JavaScript Objects?

- JavaScript is an object-based language and in JavaScript almost everything is an object or acts like an object
- A JavaScript object is just a collection of named values. These named values are usually referred to as properties of the object
- An object is similar to an array, but the difference is that you define the keys yourself, such as name, age, gender, and so on

Creating Objects

- An object can be created with curly brackets {} with an optional list of properties. A property is a "key: value" pair, where the key (or *property name*) is always a string, and value (or *property value*) can be any data type

```
var person = {  
    name: "Ali",  
    age: 28,  
    gender: "Male",  
    displayName: function() {  
        alert(this.name);  
    }  
};
```

Accessing Object's Properties

- To access or get the value of a property, you can use the dot (.), or square bracket ([]) notation, as demonstrated in the following example:

```
var book = {  
    "name": "C++",  
    "author": "Saleem",  
    "year": 2020  
};  
// Dot notation  
document.write(book.author); // Prints: saleem  
// Bracket notation  
document.write(book["year"]); // Prints: 2020
```

What is String in JavaScript

- A string is a sequence of letters, numbers, special characters and arithmetic values or combination of all.
- Strings can be created by enclosing the string literal (i.e. string characters) either within single quotes (') or double quotes ("), as shown in the example below:

Example

```
var myString = 'Hello World!'; // Single quoted string
```

```
var myString = "Hello World!"; // Double quoted string
```

Methods of String

replace()	It replaces a given string with the specified replacement.
substr()	It is used to fetch the part of the given string on the basis of the specified starting position and length.
substring()	It is used to fetch the part of the given string on the basis of the specified index.
slice()	It is used to fetch the part of the given string. It allows us to assign positive as well negative index.
toLowerCase()	It converts the given string into lowercase letter.
toUpperCase()	It converts the given string into uppercase letter.
toString()	It provides a string representing the particular object.
valueOf()	It provides the primitive value of string object.
split()	It splits a string into substring array, then returns that newly created array.
trim()	It trims the white space from the left and right side of the string.

JavaScript - The Date Object

- The Date object is a datatype built into the JavaScript language. Date objects are created with the **new Date()** as shown below.
- Once a Date object is created, a number of methods allow you to operate on it. Most methods simply allow you to get and set the year, month, day, hour, minute, second, and millisecond fields of the object, using either local time or UTC (universal, or GMT) time.

```
new Date( )
```

```
new Date(milliseconds)
```

```
new Date(datestring)
```

```
new Date(year,month,date[,hour,minute,second,millisecond ])
```

JavaScript Get Date Methods

- `getFullYear()` Get the year as a four digit number (yyyy)
- `getMonth()` Get the month as a number (0-11)
- `getDate()` Get the day as a number (1-31)
- `getHours()` Get the hour (0-23)
- `getMinutes()` Get the minute (0-59)
- `getSeconds()` Get the second (0-59)
- `getMilliseconds()` Get the millisecond (0-999)
- `getTime()` Get the time (milliseconds since January 1, 1970)
- `getDay()` Get the weekday as a number (0-6)
- `Date.now()` Get the time. ECMAScript 5.

JavaScript Set Date Methods

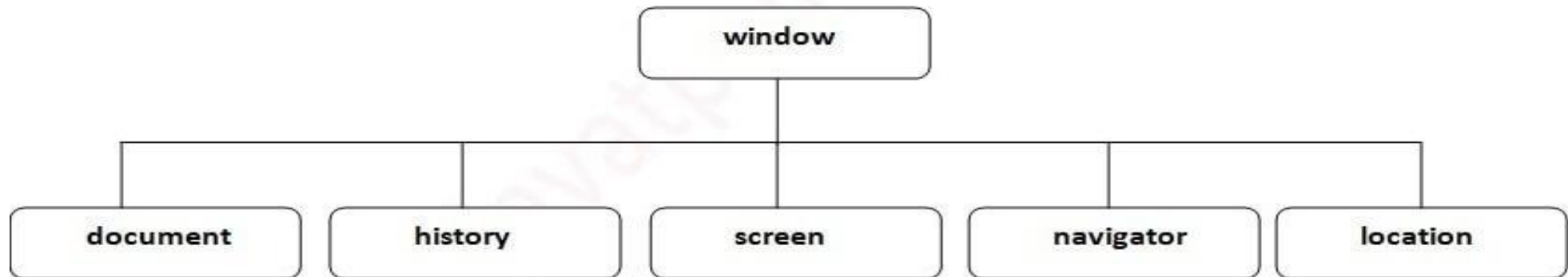
- `setDate()` Set the day as a number (1-31)
- `setFullYear(1919)` Set the year (optionally month and day)
- `setHours()` Set the hour (0-23)
- `setMilliseconds()` Set the milliseconds (0-999)
- `setMinutes()` Set the minutes (0-59)
- `setMonth()` Set the month (0-11)
- `setSeconds()` Set the seconds (0-59)
- `setTime()` Set the time

Object, String, and Date

DEMO-1

Browser Object Model

- The **Browser Object Model** (BOM) is used to interact with the browser. The default object of browser is window means you can call all the functions of window by specifying window or directly.
- For example: You can use a lot of properties (other objects) defined underneath the window object like document, history, screen, navigator, location, inner Height, inner Width,



Properties and methods of windows object

- **window.innerHeight** - the inner height of the browser window (in pixels)
- **window.innerWidth** - the inner width of the browser window (in pixels)
- **Alert()** displays the alert box containing message with ok button.
- **confirm()** displays the confirm dialog box with ok and cancel button.
- **prompt()** displays a dialog box to get input from the user.
- **open()** opens the new window.
- **close()** closes the current window.
- **setTimeout()** performs action after specified time like calling function,

The Screen Object

- The **JavaScript screen object** holds information of browser screen. It can be used to display screen width, height, colorDepth, pixelDepth etc.

Property	Description
Width	Returns the Width of the Screen
Height	Returns the Height of the Screen
Availwidth	Returns the Available Width
Availheight	Returns the Available Height
Colordepth	Returns the Color Depth
Pixeldepth	Returns the Pixel Depth.

The History Object

- The history property of the Window object refers to the History object. It contains the browser session history, a list of all the pages visited in the current frame or window.
- **Methods**
- `history.back()` - same as clicking back in the browser
- `history.forward()` - same as clicking forward in the browser
- `window.history.go(-2);` // Go back two pages
- `window.history.go(-1);` // Go back one page
- `window.history.go(0);` // Reload the current page
- `window.history.go(1);` // Go forward one page
- `window.history.go(2);` // Go forward two pages

JavaScript Navigator Object

- The **JavaScript navigator object** is used for browser detection. It can be used to get browser information such as appName, userAgent etc.

• Property	Description
• appName	returns the name
• appVersion	the version
• cookieEnabled	returns true if cookie is enabled otherwise false
• platform	returns the platform e.g. Win32.
• online	returns true if browser is online otherwise false.

Window, History, Screen, and navigation objects

DEMO-2